

- 人工智能实验报告 第9周
 - 一.实验题目：Q-Learning和Sarsa算法
 - 二.实验内容
 - 1.算法原理
 - 2.关键代码展示(可选)
 - 三.实验结果及分析
 - 1.实验结果展示示例
 - 2.评测指标展示及分析

人工智能实验报告 第9周

姓名:陈安康 学号:22320021

一.实验题目：Q-Learning和Sarsa算法

二.实验内容

1.算法原理

Q-Learning是强化学习中一种基于值的算法，其核心思想是通过一个Q表（Q-Table）来记录在每个状态下采取每个动作所能获得的期望奖励。这个Q表是一个二维表格，其中每一行代表一个状态（state），每一列代表一个动作（action），而表格中的每个值则代表在对应状态下采取对应动作所能获得的最大的未来期望奖励。

Sarsa（State-Action-Reward-State-Action）算法与Q-Learning类似，也是一种基于值的强化学习算法。但与Q-Learning不同的是，SARSA是一种在线策略迭代方法，它依赖于当前策略来选择动作，并在每次迭代中更新Q值。

具体来说，Q-Learning算法在更新当前状态时，智能体对于动作确定的下一步状态的选择策略是选择下一状态的最佳动作，而Sarsa算法是基于下一步状态根据当前策略选择动作。以下分别是Q-Learning和Sarsa的Q值更新公式，用s表示目前状态，s_表示下一个状态，a表示基于目前状态选择的动作，a_表示基于下一步状态选择的动作，α表示学习率，r表示当前奖励，γ表示奖励折扣因子：

Q-Learning:

$$Q(s, a) = Q(s, a) + \alpha * (r + \gamma * \max_{a'}(Q(s, a')) - Q(s, a))$$

Sarsa:

$$Q(s, a) = Q(s, a) + \alpha * (r + \gamma * Q(s_-, a_-) - Q(s, a))$$

在动作选择策略上，它们都使用 ϵ -greedy策略来选择动作，即在大多数情况下选择Q值最大的动作，但偶尔（以 ϵ 的概率）随机选择一个动作以进行探索。

在更新函数上，二者也有所不同，Sarsa算法在选择下一步状态更新当前状态时要求也要确定下一个状态对应的行为，即用来更新当前状态所用的行为，而Q-Learning算法并不要求这样，它只要求状态转换，并不要求因此它们两个的更新函数分开来进行设计。

在代码实现上，我将结束状态加以区分，用来统计不同结局的次数来展示训练和测试情况。

在优化上，采用对不同结局赋予不同初始奖励值的操作来进行优化。

还可以通过对探索率的动态调整来进行优化，随着训练的进行，探索率不断下降，不再趋向于对环境的探索，以收敛于最优的策略上。

2.关键代码展示(可选)

因为两个算法实现代码相差不大，所以这里只展示Q-Learning算法的代码。

在检查状态是否存在的函数中，我们将结束状态进行区分，分为“好结局”和“坏结局”。然后将坏结局的初始奖励值设为负，“好结局”的初始奖励值设为正，这样可以使训练效果

```
def check_state_exist(self, state):
    if state not in self.q_table:
        if state == 'good_terminal':
            init_actions = [5.0, 5.0, 5.0, 5.0]
            self.q_table[state] = init_actions
            return
        if state == 'bad_terminal':
            init_actions = [-5.0, -5.0, -5.0, -5.0]
            self.q_table[state] = init_actions
            return
        init_actions = [0.0, 0.0, 0.0, 0.0]
        self.q_table[state] = init_actions
```

更好：

对于探索率的动态调整实现上，将迭代次数作为参数传入，然后设置一个计数器，每一次迭代完成后，计数器加一，依据计数器和迭代次数对探索率进行计算，从而实现探索率的动态调整：

```
def __init__(self, actions, learning_rate=0.01, reward_decay=0.9, e_greedy=0.9, episode = 50):
    self.actions = actions # a list
    self.lr = learning_rate
    self.gamma = reward_decay
    self.epsilon = e_greedy
    self.episode = episode
    self.times = 0
```

```
for episode in range(RL.episode):
    # initial observation
    observation = env.reset()
    # action = random.choice(RL.actions)
    # action = RL.choose_action(str(observation))
    while True:
        RL.times += 1
        env.render()
        action = RL.choose_action(str(observation))
```

```
def choose_action(self, observation):
    self.check_state_exist(observation)
    if np.random.rand() < self.epsilon * (1 - float(self.times) / self.episode):
        action = np.random.choice(self.actions)
    else:
        action = self.get_max_Q_action(self.q_table[observation])
    return action
```

三.实验结果及分析

1.实验结果展示示例

将训练次数设为50，同时测试50次，结果如下，其中good表示智能体达到黄色圆圈的次数，bad表示智能体达到黑色方框的次数：

Q-Learning:

```
PS C:\Users\86135\Desktop\qlearning&sarsa> & "C:/Program Files/Python37/python.exe" C:/Users/86135/Desktop/qlearning&sarsa/qlearning.py
Q-Learning: good = 43 | bad = 7 | normal = 0
train over
good = 50 | bad = 0 | normal = 0
game over
PS C:\Users\86135\Desktop\qlearning&sarsa>
```

Sarsa:

```

● PS C:\Users\86135\Desktop\qlearning&sarsa> & "C:\Users\86135\Desktop\qlearning&sarsa\test.py"
Sarsa: good = 42 | bad = 8 | normal = 0
train over
good = 50 | bad = 0 | normal = 0
game over
● PS C:\Users\86135\Desktop\qlearning&sarsa>

```

从打印信息可以很清楚看到，测试集均可全部到达黄色圆圈，成功率100%。

加入路径打印模块后，打印得到的部分路径展示如下，因为到达终点时（无论好坏）迷宫传出的不再是坐标信息，而是终结信号，因此只能打印字符串来进行标识：

Q-Learning:

```

Q-Learning: good = 44 | bad = 6 | normal = 0
train over
[[0, 0], [0, 1], [0, 2], [0, 2], [0, 2], [0, 2], [0, 1], [0, 2], [0, 1], [0, 2], [0, 2], [0, 2], [0, 1], [0, 2], [0, 3], [1, 3], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 2], [0, 1], [0, 2], [0, 2], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 2], [0, 1], [0, 2], [0, 1], [0, 2], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 1], [0, 2], [0, 1], [0, 2], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 2], [0, 2], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']

```

```

[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 2], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 1], [0, 2], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 2], [0, 2], [0, 2], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
good = 50 | bad = 0 | normal = 0
game over
PS C:\Users\86135\Desktop\qlearning&sarsa>

```

Sarsa:

```

Sarsa: good = 44 | bad = 6 | normal = 0
train over
[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 1], [0, 2], [0, 2], [0, 1], [0, 2], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 2], [0, 1], [0, 2], [0, 2], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 1], [0, 2], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']

```

```

[[0, 0], [0, 1], [0, 2], [0, 1], [0, 2], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 1], [0, 2], [0, 1], [0, 2], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 1], [0, 2], [0, 2], [0, 3], [1, 3], [1, 3], [2, 3], 'good']
[[0, 0], [0, 1], [0, 2], [0, 3], [1, 3], [2, 3], 'good']
good = 50 | bad = 0 | normal = 0
game over
PS C:\Users\86135\Desktop\qlearning&sarsa>

```

2.评测指标展示及分析

评测指标主要为成功率，即智能体成功到达黄色圆圈的概率。

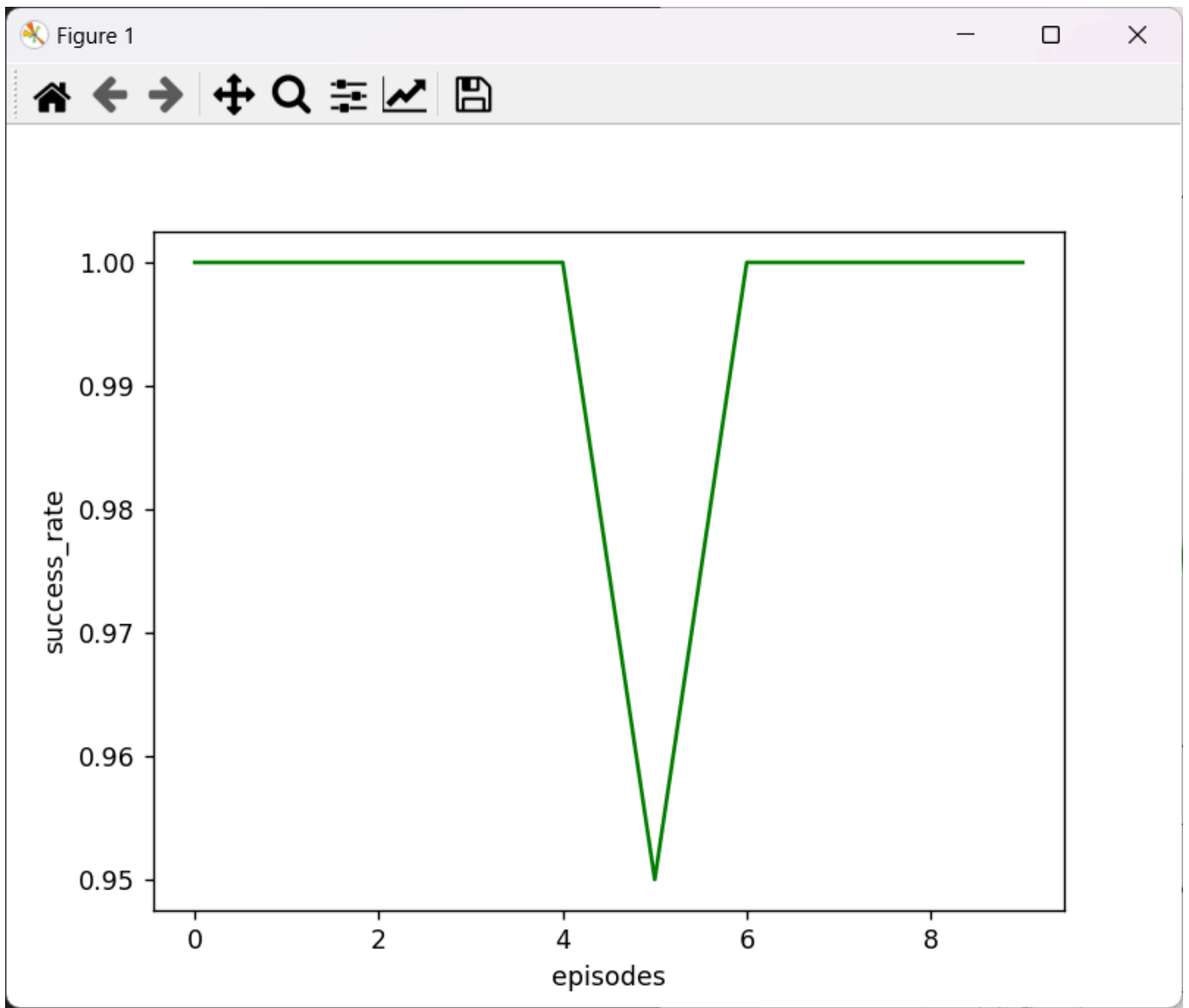
以下都是基于训练迭代次数设置为50的前提下进行的。

为了直观展示这一指标，修改代码如下，我们每组测试20次，一共10组，累积为200次，每组统计成功率进行画图展示：

```
RL = QLearning(actions=list(range(env.n_actions)))
# RL = Sarsa(actions=list(range(env.n_actions)))
update_q_learning()
# update_sarsa()
success_rate = []
for i in range(10):
    g,b = test()
    success_rate.append(float(g) / (g + b))
plt.plot(range(len(success_rate)),success_rate, color = 'g')
plt.xlabel("episodes")
plt.ylabel("success_rate")
plt.show()
env.destroy()
# env.after(100, test)
# env.mainloop()
```

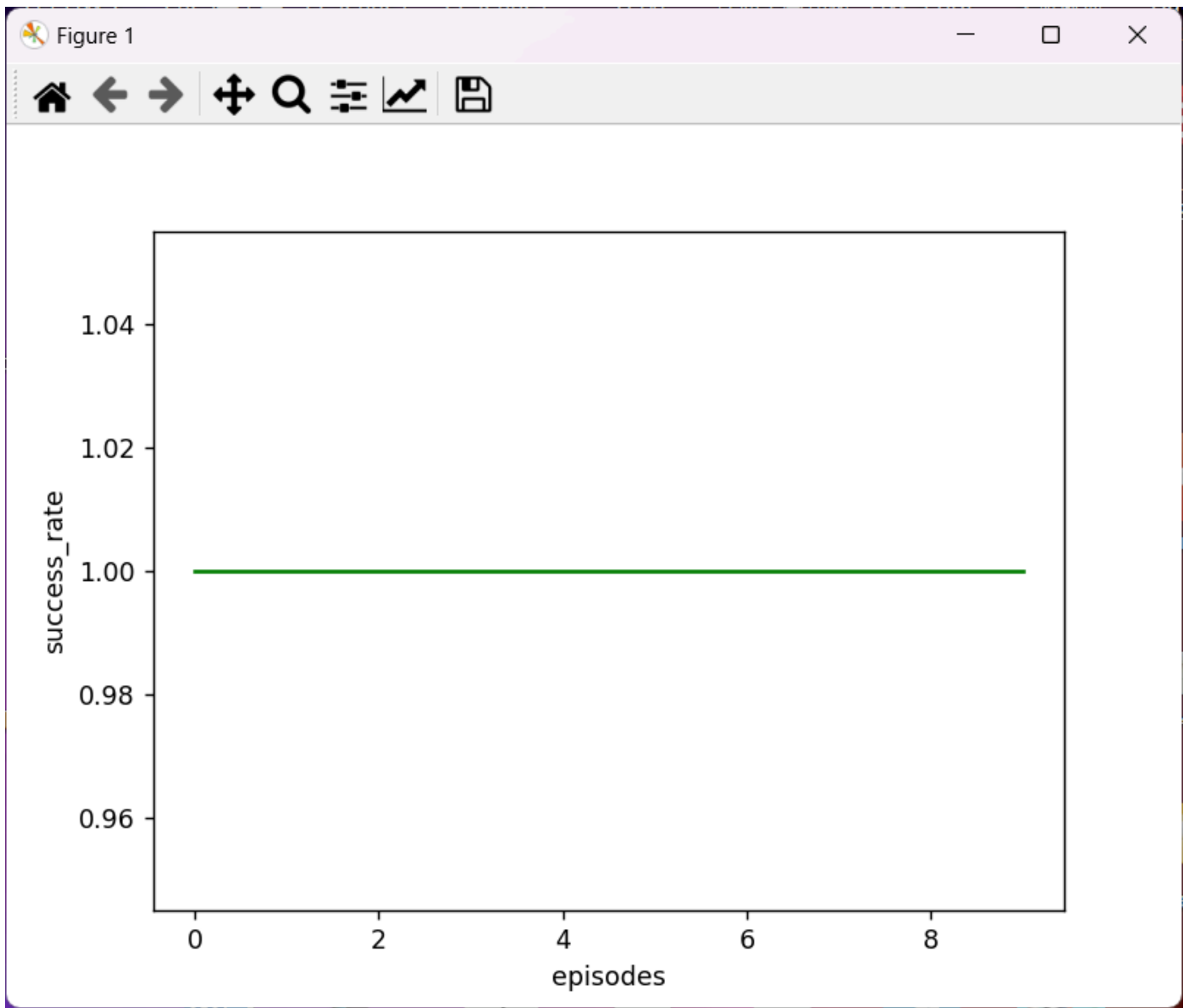
展示结果如下：

Q-Learning:



平均成功率为99.5%。

Sarsa:



平均成功率为100%

因为在当前状态下，Q值高的动作被选择的几率更大，而不是一定被选择，所以这个结果是十分理想的。